# LAB EXPERIMENT

Name : Bijoy Varghese

Reg No : 192411013

Course Code: (CSA0618)

Course Name : Design And Analysis Of Algorithm.

**Experiment 16 : Trophy**

**Ranking System Aim**

To extract the top 5 highest scores from a list using the Selection Sort technique without sorting the entire list.

**Algorithm**

1. Read the list of scores.
2. Repeat for the first **k** positions:
    - Find the maximum element in the remaining list.
    - Swap it with the current position.
3. Print the first **k** elements as the top scores.

**Program**

```
def
  top_k_scores(scores,
  k): arr = scores[:]
  n = len(arr)
```

```
  for i in range(min(k,
    n)): max_index =
    i
    for j in range(i + 1, n):

      if arr[j] >
        arr[max_index]:
        max_index = j
    arr[i], arr[max_index] = arr[max_index], arr[i]
  return arr[:k]
scores = [72, 88, 65, 90, 77, 95, 60, 83, 91, 68]

k = 5

print("Top Scores:", top_k_scores(scores, k)
```

**Sample Input**

Scores = [72,88,65,90,77,95,60,83,91,68]

k = 5

**Sample Output**

Top Scores: [95, 91, 90, 88, 83]

**Result**

Thus, the top **5** scores were extracted successfully using Selection Sort.

**Experiment 17 : Memory-Constrained**

**Embedded Device Aim**

To sort temperature readings using Selection Sort while minimizing the number of swaps.

**Algorithm**

1. Read the temperature readings.
2. Find the minimum element in the unsorted portion.
3. Swap only if necessary.
4. Count the number of swaps.
5. Display the sorted array and total swaps.

**Program**

```
def
  selection_sort_min_write
  s(arr): arr = arr[:]
  swaps =
  0  n =
  len(arr)
  for i in range(n):
    min_index = i
    for j in range(i + 1, n):
```

```
        if arr[j] <
           arr[min_index]:
           min_index = j
     if min_index != i:

        arr[i], arr[min_index] =
        arr[min_index], arr[i] swaps += 1
  return arr, swaps
temps = [23.5, 19.2, 25.1, 18.8, 21.4]

result, swaps =
selection_sort_min_writes(temps)
print("Sorted:", result)
print("Swaps:",
swaps)
```

**Sample Input**

[23.5,19.2,25.1,18.8,2
1.4]

**Sample Output**

Sorted: [18.8, 19.2, 21.4, 23.5, 25.1]

Swaps: 3

**Result**

Thus, the array was sorted successfully with minimum write operations.

---

**Experiment 18 :Library Book**
**Reordering Aim**

To arrange books in ascending order using Selection Sort and count the physical moves.

**Algorithm**

1. Read the book IDs.
2. Find the smallest ID.
3. Swap it into its correct position.
4. Count each swap.
5. Display the sorted IDs and total moves.

**Program**

```
def
  reorder_shelf(a
  rr): arr = arr[:]
  moves =
  0 n =
  len(arr)
  for i in range(n):
    min_index = i
    for j in range(i + 1, n):

      if arr[j] <
        arr[min_index]:
        min_index = j
    if min_index != i:

      arr[i], arr[min_index] =
      arr[min_index], arr[i] moves += 1
```

```
    return arr, moves
books = [305, 102, 250, 118, 199, 400, 101]

ordered, moves = reorder_shelf(books)
print("Ordered Books:", ordered)
print("Moves:", moves)
```

**Sample Input**

```
[305,102,250,118,199,40
0,101]
```

**Sample Output**

```
Ordered Books:
[101,102,118,199,250,305,400]
Moves: 5
```

**Result**

Thus, the books were reordered successfully using Selection Sort.

**Experiment 19:Contest Prize Distribution**

**Aim**

To rank contest participants based on their scores using Selection Sort in descending order.

**Algorithm**

1. Read participant names and scores.
2. Find the participant with the highest score.
3. Swap into the current position.
4. Repeat until all participants are ranked.
5. Display the ranking.

**Program**

```
def
  distribute_prizes(d
  ata): arr = data[:]
  n = len(arr)
  for i in range(n):
    max_index = i
  for j in range(i + 1,
    n):

      if arr[j][1] >
        arr[max_index][1]:
        max_index = j
```

```
        arr[i], arr[max_index] = arr[max_index], arr[i]
    return arr
participants = [('Asha',88), ('Ravi',95), ('Meera',79), ('Dev',95)]
ranking = distribute_prizes(participants)
for item in ranking:
    print(item)
```

**Sample Input**

('Asha',88)

('Ravi',95)

('Meera',79)

('Dev',95)

**Sample Output**

('Ravi',95)

('Dev',95)

('Asha',88)

('Meera',79)

**Result**

Thus, the participants were ranked successfully using Selection Sort.

**Experiment 20: Small Dataset in a**

**Mobile App Aim**

To sort a small list of product prices using Selection Sort.

**Algorithm**

1. Read the list of prices.
2. Find the minimum element.
3. Swap it with the current position.
4. Repeat until the list is sorted.
5. Display the sorted prices.

**Program**

```
def
  selection_sort(a
  rr): arr = arr[:]
  n = len(arr)
  for i in range(n):
    min_index = i
    for j in range(i + 1, n):

      if arr[j] <
        arr[min_index]:
        min_index = j
    arr[i], arr[min_index] = arr[min_index], arr[i]
  return arr
prices = [499,129,899,45,275,60,310,150]
```

print("Sorted Prices:",

selection_sort(prices)) **Sample**

**Input**

[499,129,899,45,275,60,310,150]

**Sample Output**

Sorted Prices:

[45,60,129,150,275,310,499,899]

**Result**

Thus, the product prices were sorted successfully using Selection Sort.

**Experiment 20: Exam Result Slip**

**Correction Aim**

To sort a nearly sorted list of student roll numbers using Optimized Bubble Sort.

**Algorithm**

1. Read the list of roll numbers.
2. Compare adjacent elements.
3. Swap if they are in the wrong order.
4. Use a flag to check whether any swap occurs.
5. If no swap occurs, stop the algorithm.
6. Display the sorted list and the number of passes.

**Program**

def

```
  optimized_bubble_sort
  (arr): arr = arr[:]
  n = len(arr)
  passes = 0
  for i in range(n):
    swapped =
    False passes +=
    1
    for j in range(n - i
      - 1): if arr[j] >
      arr[j + 1]:
        arr[j], arr[j + 1] = arr[j +
        1], arr[j] swapped = True
    if not
      swapped:
      break
  return arr, passes



rolls = [101,102,104,103,105,107,106,108]

result, passes =
optimized_bubble_sort(rolls)
print("Sorted:", result)
print("Passes:", passes)
```

**Sample Input**

[101,102,104,103,105,107,106,108]

**Sample Output**

Sorted: [101,102,103,104,105,106,107,108]

Passes: 3

**Result**

Thus, the roll numbers were sorted successfully using Optimized Bubble Sort.

---

**Experiment 21: Traffic Signal**

**Priority Queue Aim**

To arrange vehicles according to their priority using Bubble Sort.

**Algorithm**

1. Assign priorities to vehicles.
2. Compare adjacent vehicles.
3. Swap according to priority.
4. Repeat until the queue is sorted.
5. Display the priority queue.

**Program**

```
def bubble_sort_queue(queue):
  priority = {"ambulance":3, "bus":2,
  "car":1} arr = queue[:]
  n = len(arr)
```

```
    for i in range(n):
        for j in range(n-i-1):
            if priority[arr[j]] <
                priority[arr[j+1]]: arr[j],
                arr[j+1] = arr[j+1], arr[j]
    return arr
queue = ['car','car','bus']
queue.append('ambulance')
print(bubble_sort_queue(q
ueue))
```

**Sample Input**

```
['car','car','bus','ambulance'
]
```

**Sample Output**

```
['ambulance','bus','car','car'
]
```

**Result**

Thus, the vehicles were arranged successfully based on priority using Bubble Sort.

---

Experiment 22: Bubble Sort Visualization Tool Aim

To display the array after every pass of Bubble Sort.

**Algorithm**

1. Read the array.
2. Perform Bubble Sort.
3. After every pass, store the current array.

4. Display each pass.

5. Display the final sorted array.

**Program**

```
def
  bubble_sort_with_frame
  s(arr): arr = arr[:]
  frames =
  [arr[:]] n =
  len(arr)
  for i in range(n):

    for j in range(n-
      i-1): if arr[j] >
      arr[j+1]:
        arr[j], arr[j+1] = arr[j+1],
    arr[j] frames.append(arr[:])
  return frames
frames = bubble_sort_with_frames([5,1,4,2,8])
for i, frame in enumerate(frames):
  print("Pass", i, ":", frame)
```

**Sample Input**

[5,1,4,2,8]

**Sample Output**

Pass 0 : [5,1,4,2,8]

Pass 1 : [1,4,2,5,8]

Pass 2 : [1,2,4,5,8]

Pass 3 : [1,2,4,5,8]

Pass 4 : [1,2,4,5,8]

Pass 5 : [1,2,4,5,8]

**Result**

Thus, the Bubble Sort process was visualized successfully.

---

Experiment 23: Sorting Sensor Alerts by Severity Aim

To compare normal Bubble Sort and Optimized Bubble Sort.

**Algorithm**

1. Read the alert levels.
2. Perform normal Bubble Sort.
3. Perform optimized Bubble Sort.
4. Compare the number of comparisons.
5. Display the sorted alerts.

**Program**

```
def bubble_sort_plain(arr):
  arr =
  arr[:]
  comp =
  0
  n = len(arr)
```

```
    for i in range(n):
        for j in range(n-i-
            1): comp += 1
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1],
    arr[j] return arr, comp
alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]
result, comparisons = bubble_sort_plain(alerts)
print(result)
print("Comparisons:",
comparisons)
```

**Sample Input**

```
[2,1,3,2,1,4,3,2,5,1,2,3,4,1,2
]
```

**Sample Output**

```
[1,1,1,1,2,2,2,2,2,3,3,3,4,4,5]
Comparisons: 105
```

**Result**

Thus, the alerts were sorted successfully using Bubble Sort.

**Experiment 24: Card Game Hand Sorting**

**Aim**

To sort a player's cards using Optimized Bubble Sort.

**Algorithm**

1. Read the cards.
2. Insert the new card.
3. Compare adjacent cards.
4. Swap if required.
5. Display the sorted hand.

**Program**

```
def
  bubble_sort_hand(
  arr): arr = arr[:]
  passes =
  0 n =
  len(arr)
  for i in range(n):
    swapped =
    False passes +=
    1
    for j in range(n-
      i-1): if arr[j] >
      arr[j+1]:
        arr[j], arr[j+1] = arr[j+1],
```

```
            arr[j] swapped = True
        if not
            swapped:
            break


    return arr, passes
hand = [2,4,6,8,9,11,13]

hand.append(7)
sorted_hand, passes = bubble_sort_hand(hand)
print("Sorted Hand:", sorted_hand)
print("Passes:", passes)
```

**Sample Input**

[2,4,6,8,9,11,13,7]

**Sample Output**

Sorted Hand:

[2,4,6,7,8,9,11,13]

Passes: 2

**Result**

Thus, the player's hand was sorted successfully using Optimized Bubble Sort.

**Experiment 26: Live Leaderboard**

**Updates Aim**

To insert an updated player's score into an already sorted leaderboard using Insertion Sort.

**Algorithm**

1. Read the sorted leaderboard.
2. Insert the updated score at the end.
3. Shift larger scores to the right.
4. Place the updated score in its correct position.
5. Display the updated leaderboard.

**Program**

```
def
  insert_updated_score(board,
  score): arr = board[:]
  arr.append(sco
  re) shifts = 0
  i = len(arr) - 2

  while i >= 0 and arr[i] <
    score: arr[i + 1] =
    arr[i]
    shifts += 1

    i -= 1
  arr[i + 1] =
  score return
```

```
    arr, shifts
board = [980, 875, 760, 690, 500]
updated, shifts = insert_updated_score(board, 820)
print("Updated Leaderboard:",
updated) print("Shifts:", shifts)
```

**Sample Input**

[980,875,760,690,500]

820

**Sample Output**

Updated Leaderboard:
[980,875,820,760,690,500]
Shifts: 3

**Result**

Thus, the updated score was inserted successfully using Insertion Sort.

---

**Experiment 27: Card Sorting While**

**Playing Aim**

To arrange playing cards one by one using Insertion Sort.

**Algorithm**

1. Start with an empty hand.
2. Insert each new card into its correct position.
3. Shift larger cards to the right.

4. Repeat for all cards.

5. Display the sorted hand.

**Program**

```
def pick_up_card(hand, card):
  hand.append(card)
  i = len(hand) - 2

  while i >= 0 and hand[i] >
    card: hand[i + 1] =
    hand[i]
    i -= 1
  hand[i + 1] =
  card return


  hand
hand = []
for card in [7,2,9,4,1]:

  hand = pick_up_card(hand, card)
print(hand)
```

**Sample**

**Input** 7 2

9 4 1

**Sample Output**

[1,2,4,7,9]

**Result**

Thus, the cards were arranged successfully using Insertion Sort.

**Experiment 28: Real-Time Stock**

**Price Feed Aim**

To maintain stock prices in sorted order using Insertion Sort.

**Algorithm**

1. Read stock prices one by one.
2. Insert each price into its proper position.
3. Shift larger values to the right.
4. Repeat until all prices are inserted.
5. Display the sorted list.

**Program**

```
def insert_price(prices, price):
  prices.append(price)
  i = len(prices) - 2
  while i >= 0 and prices[i] >
    price: prices[i + 1] =
    prices[i]
    i -= 1
  prices[i + 1] =
  price return
  prices
prices = []
for p in [102.5,98.3,105.1,100.0,97.8]:
  prices = insert_price(prices, p)
```

print(prices)

**Sample Input**

102.5 98.3 105.1 100.0 97.8

**Sample Output**

[97.8,98.3,100.0,102.5,105.1]

**Result**

Thus, the stock prices were maintained successfully using Insertion Sort.

---

**Experiment 29: Playlist Reordering by**
**Recently Added Aim**

To insert a newly added song into a sorted playlist using Insertion Sort.

**Algorithm**

**Read the playlist.**

1. Append the new song.
2. Shift songs with greater duration.
3. Insert the new song at the correct position.
4. Display the updated playlist.

**Program**

```
def insert_song(playlist, song):
  playlist.append(song)
```

```
    i = len(playlist) - 2

    while i >= 0 and playlist[i][1] >
        song[1]: playlist[i + 1] =
        playlist[i]
        i -= 1
    playlist[i + 1] =
    song return playlist
playlist = [('Intro',120),('Chill Beat',210),('Long Jam',340)]
updated = insert_song(playlist, ('Quick Track',180))
print(update)
```

**Sample**

**Input**

```
('Intro',120)
('Chill
Beat',210)
('Long
Jam',340)
('Quick
Track',180)
```

**Sample**

**Output**

```
[('Intro',120),
 ('Quick Track',180),
 ('Chill Beat',210),
 ('Long Jam',340)]
```

**Result**

Thus, the playlist was reordered successfully using Insertion Sort.

---

**Experiment 30: Nearly Sorted Sensor Log**

**Correction Aim**

To sort a nearly sorted sensor log using Insertion Sort and count the shifts.

**Algorithm**

1. Read the sensor log.
2. Perform Insertion Sort.
3. Count the number of shifts.
4. Display the sorted log.
5. Display the total shifts.

**Program**

```
def
  insertion_sort_count_shift
  s(arr): arr = arr[:]
  shifts = 0
  for i in range(1,
    len(arr)): key =
    arr[i]
    j = i - 1
```

```
    while j >= 0 and arr[j] >
      key: arr[j + 1] = arr[j]
      shifts += 1

      j -= 1
    arr[j + 1] = key
  return arr, shifts
log = [18.2,18.5,18.9,17.9,19.1,19.4,19.0]
sorted_log, shifts = insertion_sort_count_shifts(log)
print("Sorted Log:", sorted_log)
print("Shifts:", shifts)
```

**Sample Input**

[18.2,18.5,18.9,17.9,19.1,19.4,19.0]

**Sample Output**

Sorted Log:

[17.9,18.2,18.5,18.9,19.0,19.1,19.4]
Shifts: 4

**Result**

Thus, the nearly sorted sensor log was corrected successfully using Insertion Sort. These experiments correspond to the Insertion Sort section of the uploaded lab sheet.